

Final Research Base and Product Requirements Document for an AI/ML Dashcam Road-Damage Analysis Project

Executive summary

The project should be treated as an **academic video-analytics system**, not merely as a YOLO pothole detector. The strongest defensible project is:

AI-Based Multi-Class Road Damage Detection, Temporal Tracking, Unique Damage Counting, Visual Severity Analysis, and Automated Reporting from Dashcam Video

The core research pipeline should be:

Dashcam video → detection → tracking → temporal stabilization → unique-event aggregation/counting → visual severity/extent estimation → timestamped reporting → annotated video/dashboard.

Your existing Perplexity research document already identifies most of these pieces—including detection, tracking, duplicate removal, severity, reporting, evaluation metrics, and a proposed architecture—but it functions more as a broad literature/idea survey than as an executable research protocol. In particular, the class taxonomy, video-evaluation protocol, tracking ground truth, unique-count definition, severity definition, reproducibility rules, and acceptance criteria need to be fixed before code is written.

The LinkedIn **RoadGuardian AI** project is useful as a practical reference implementation: it combines a custom YOLO26 detector, OpenCV video processing, ByteTrack persistent IDs, temporal smoothing, an anti-flicker frame-hold mechanism, and video-in/video-out processing. It is therefore a good **engineering baseline**, but it does not by itself establish the research methodology needed for your academic project: there is no published unique-damage ground-truth protocol, tracker comparison, counting accuracy study, multi-class RDD2022 evaluation, severity validation, or controlled ablation in the post. ¹

The most important design decision is to create **two different custom datasets from your teacher's one-hour dashcam video**:

Dataset	Purpose	Sampling
Detection dataset	Train/validate YOLO	Sparse, diverse frames

Dataset	Purpose	Sampling
Video tracking/counting benchmark	Evaluate tracking, ID switches and unique counts	Consecutive video frames/clips

These must not be conflated. Sparse 1 FPS frames can be excellent for annotation and detector training, but multi-object tracking requires temporally consecutive frames. Current Ultralytics tracking maintains state across consecutive frames, and its documentation explicitly requires persistent sequential input to preserve track IDs. ²

The canonical V1 taxonomy should be the four RDD2022 challenge classes:

ID	RDD code	Project label
0	D00	Longitudinal crack
1	D10	Transverse crack
2	D20	Alligator crack
3	D40	Pothole

The official RDD2022 article describes 47,420 images from six countries and more than 55,000 road-damage instances belonging to these four categories, while the official RoadDamageDetector repository likewise specifies D00, D10, D20 and D40 as the categories considered for CRDDC2022. ³ Your teacher video's actual class coverage is **currently unspecified**. The project must not manufacture absent classes; public RDD2022 data can provide training examples, while local evaluation should state `N/A` for a class with no custom-video ground truth.

For detection, compare **YOLO26s against YOLOv8s** under a common experimental protocol. Ultralytics currently documents YOLO26 as its January 2026 model family, with end-to-end inference and detection, segmentation and other task variants; YOLOv8 remains officially supported and is therefore a useful historical baseline. ⁴ If GPU memory is insufficient, switch *both* experiments to the nano variants rather than comparing mismatched model sizes.

For tracking, compare **BoT-SORT against ByteTrack**, explicitly selecting the tracker configuration rather than relying on framework defaults. Current Ultralytics documentation recommends ByteTrack as the simple/fast baseline and BoT-SORT for moving-camera footage because BoT-SORT supports camera-motion compensation and optional ReID. The current Ultralytics default is no longer necessarily one of these—it documents TrackTrack as the default—so the experiments must explicitly specify `botsort.yaml` and `bytetrack.yaml`. ² This is particularly relevant because BoT-SORT's original design combines motion, appearance information and camera-motion compensation, whereas ByteTrack's primary contribution is association of lower-confidence detections to reduce fragmented trajectories. ⁵

The final project should **not claim physical pothole depth, volume, or engineering-grade severity** from an uncalibrated monocular dashcam. V1 should report a clearly named **Visual Severity / Visual Extent Tier**, measured in a standardized region of the road view to reduce perspective bias. Your original report already recognizes the limitation of absolute monocular size/depth estimates. `filecite` `turn0file0`

The recommended final source-of-truth document is:

docs/PROJECT_PRD.md

The remainder of this report is suitable to become its first version.

Audit of the prior research and design

The prior report contains a strong project concept, but the following issues should be resolved **before implementation**. [filecite turns file into](#)

Priority	Weak point	Why it matters	Required correction
P0	RDD2022 taxonomy is inconsistent in the prior report	Parts of the document describe seven/eight classes, while the official CRDDC/RDD2022 research task uses four primary damage categories	Freeze V1 to D00, D10, D20, D40
P0	No tracking ground truth is defined	ID switches, IDF1/HOTA and tracker quality cannot be calculated without human track identities	Create a short manually tracked video benchmark
P0	Unique count is too closely equated with tracker IDs	One physical defect may receive multiple IDs; conversely, one ID can occasionally absorb multiple detections	Implement a separate <code>DamageEventAggregator</code> above the tracker
P0	Sparse frame sampling and tracking are mixed conceptually	1–5 FPS can be appropriate for dataset creation but may destroy temporal association during tracking	Separate annotation sampling FPS from runtime/tracking FPS
P1	Severity = bounding-box area is perspective-biased	The same pothole becomes much larger in pixels as the vehicle approaches	Measure at a standardized image/road zone and call it visual extent
P1	A single area metric is inappropriate across all four classes	Long cracks can enclose large empty bounding-box regions while potholes are more object-like	Use class-specific visual-extent features or restrict strong severity claims

Priority	Weak point	Why it matters	Required correction
P1	The prior motion description is oversimplified	In a typical forward-facing camera, road-plane points generally move from around the vanishing region downward/outward as the vehicle approaches them; camera pitch and vibration alter this	Learn/tune counting trajectories from the actual video rather than hard-code the wrong direction
P1	Success metrics have names but no operational definitions	“Unique-count accuracy” and “FPS” are meaningless unless their formulas and benchmark hardware are frozen	Define formulas, datasets and reference hardware in the PRD
P1	Temporal leakage prevention is not strict enough	Adjacent video frames are strongly correlated and can inflate held-out performance	Split by temporal/route blocks and physical defect event, never by random frame
P1	No annotation-QA protocol	Label inconsistency can dominate detector error	Define label rules, double-review subset and automated label validation
P1	No experiment/version provenance	Results cannot be reproduced if model version, split, seed, tracker config and commit are lost	Every run gets a manifest
P1	Privacy/licensing is unresolved	Teacher-video redistribution rights are unknown and public frames may contain plates/faces	Treat teacher footage as private until permission is documented
P2	Road ROI is treated as automatically beneficial	An aggressive crop can hide valid road damage	Make ROI configurable and initially use it as a post-detection filter
P2	Dashboard appears too early in the architecture	UI work can consume time before the research pipeline works	Build dashboard only after metrics and full-video processing pass
P2	Too many possible extensions	Segmentation, GPS, homography, depth and mobile deployment can dilute the main contribution	Explicitly move them to future work

The RDD taxonomy correction is especially important. The official RDD2022 paper describes four damage types, and the official repository explicitly lists `{D00, D10, D20, D40}` for the CRDDC task. The public train data is annotated in Pascal VOC XML; the official challenge test images were released without

annotations, so your project cannot simply use the official RDD2022 challenge test set for local mAP calculation. ⁶

A second important methodological correction concerns leakage. Consecutive video frames are inherently highly correlated; video-learning research explicitly exploits that temporal coherence, and other video-ML studies avoid leakage by assigning all correlated frames from the same source video/segment to a single split. ⁷ For your project, the unit of splitting should therefore be the **road/video block and physical damage event**, not the individual JPEG.

Final product requirements document

Suggested filename: docs/PROJECT_PRD.md

Product definition

The system shall accept a forward-facing dashcam video and produce:

```
input_video.mp4
  ↓
road-damage analysis
  ↓
annotated_video.mp4
damage_events.csv
damage_events.json
run_metrics.json
optional_dashboard/
```

Each confirmed damage event should minimally contain:

```
{
  "event_id": "RD000123",
  "class_code": "D40",
  "class_name": "pothole",
  "first_seen_ms": 731240,
  "last_seen_ms": 733920,
  "representative_timestamp_ms": 732400,
  "mean_confidence": 0.87,
  "visual_severity": "medium",
  "source_video": "teacher_dashcam_01.mp4"
}
```

Goals

The primary goal is to determine whether a modern object detector combined with temporal tracking and event aggregation can convert dashcam footage into a more reliable **unique road-damage inventory** than independent frame-by-frame detection.

The academic goals are to:

1. build a four-class RDD2022-compatible road-damage detector;
2. adapt it to the teacher-provided dashcam domain;
3. compare YOLO26 and YOLOv8;
4. compare BoT-SORT and ByteTrack;
5. measure whether temporal smoothing improves visual/temporal stability;
6. measure whether event aggregation reduces repeated counting;
7. produce timestamps, visual severity/extent and machine-readable reports;
8. evaluate the complete system on held-out custom footage.

RDD2022 is particularly appropriate as the public training basis because it contains data from six countries including India, was collected from vehicle/smartphone-camera settings, and intentionally includes variations in environment and illumination. ⁸

Scope

In scope for V1	Out of scope for V1
Four-class object detection	True pothole depth
RDD2022 transfer learning	Physical volume estimation
Custom dashcam fine-tuning	Engineering pavement-condition certification
YOLO26 vs YOLOv8	Full semantic/instance segmentation
BoT-SORT vs ByteTrack	LiDAR/stereo reconstruction
Temporal smoothing	Full visual odometry/SLAM
Track confirmation	Precise physical dimensions
Unique-event counting	Production-grade mobile application
Relative visual severity/extent	Autonomous vehicle control
Timestamps/frame numbers	GPS map unless GPS data is later provided
CSV/JSON reporting	Cloud-scale infrastructure
Annotated MP4	Commercial deployment
Basic demo dashboard	Large enterprise dashboard
Hard-negative mining	Patch/repair classes in V1

The LinkedIn RoadGuardian implementation demonstrates that YOLO26 + ByteTrack + temporal smoothing + frame-hold visualization is technically viable as an engineering pipeline; your project's differentiator should therefore be **multi-class detection, formal tracking/count ground truth, tracker comparison, unique-event aggregation, severity/report generation and controlled experiments**, rather than simply reproducing its interface. ¹

Assumptions and unresolved inputs

Anything marked **unspecified** must be captured during the video-audit milestone rather than guessed.

Parameter	Status / assumption
Teacher video duration	Approximately one hour, user-provided
Video resolution	Unspecified
Source FPS	Unspecified
Codec/bit rate	Unspecified
Camera mounting height/angle	Unspecified
Day/night/weather conditions	Unspecified
Class distribution in teacher video	Unspecified
Presence of all four RDD classes	Unspecified
GPS metadata	Not assumed
Camera calibration	Not assumed
Available local GPU	Unspecified
Cloud GPU availability	Unspecified
Storage available	Unspecified
Academic deadline	Unspecified
Teacher permission to redistribute video/frames	Unspecified
Physical severity ground truth	Not assumed
RDD taxonomy	D00/D10/D20/D40
Primary annotation geometry	Bounding boxes
Primary model scale	☐ s, falling back to ☐ n for both detectors if required

Constraints

The principal constraint is annotation labor. A one-hour 30 FPS video would contain roughly 108,000 frames, so annotating every frame would be unnecessary and inefficient. The public RDD2022 dataset mitigates

some training-data requirements, but its official test portion has no labels, meaning quantitative local evaluation still requires your own held-out annotations. ⁸

The second constraint is that general-purpose multi-object trackers were designed around conventional tracked objects. ByteTrack associates high- and lower-confidence detections to preserve trajectories, while BoT-SORT adds camera-motion compensation and optional appearance cues; neither paper specifically establishes perfect behavior for stationary pavement defects viewed from a moving camera. Therefore tracking must be evaluated rather than assumed to solve duplicate counting. ⁹

Success metrics

The following are **provisional project acceptance targets**, not claims about literature performance. They should be frozen after the first dataset audit and before the final experiment.

Dimension	Metric	MVP floor	Preferred target	Evaluation set
Detection	mAP@0.5	≥ 0.50	≥ 0.60	Custom temporal test
Detection	mAP@0.5:0.95	≥ 0.30	≥ 0.35	Custom temporal test
Detection	Precision / Recall	Report	Report	Per class
Counting	Unique-count accuracy	≥ 0.85	≥ 0.90	Tracking/count benchmark
Counting	Event-level F1	≥ 0.80	≥ 0.85	Tracking/count benchmark
Tracking	Raw ID switches	Report	Minimize	Tracking benchmark
Tracking	IDSW / 100 GT tracks	≤ 15	≤ 10	Tracking benchmark
Tracking	IDF1	Report	Higher than alternative	Tracking benchmark
Tracking	HOTA	Report	Higher than alternative	Tracking benchmark
Performance	End-to-end FPS	Report until GPU frozen	≥ 15 FPS on frozen reference GPU	Fixed benchmark clip
Reliability	One-hour processing	No crash	No crash + all outputs valid	Teacher video

HOTA, IDF1, MOTA and ID-switch statistics can be generated using the TrackEval ecosystem, whose reference implementation explicitly supports HOTA, CLEAR MOT and identity metrics. ¹⁰ CVAT can export

MOT-formatted bounding-box tracks, making it practical to construct the required custom evaluation sequence. ¹¹

Unique-count accuracy should be defined rather than left ambiguous:

$$UCA = \max \left(0, 1 - \frac{|N_{pred} - N_{gt}|}{\max(N_{gt}, 1)} \right)$$

However, this metric alone is insufficient because an equal number of false positives and misses could cancel numerically. Therefore the project must also compute **event-level precision, recall and F1**, where a predicted event is matched to a manually labeled physical defect using class and temporal/spatial overlap.

FPS must mean:

Frames processed / total wall-clock processing time, including decoding, detector inference, tracker and event aggregation, excluding interactive dashboard rendering.

A second “with-video-rendering FPS” may be reported separately.

Milestones and acceptance criteria

Milestone	Output	Acceptance criterion
Requirements freeze	PRD + project files	Scope, classes, metrics and naming frozen
Video audit	<code>video_profile.json</code>	Resolution/FPS/duration/codec and quality documented
Annotation pilot	100–200 reviewed frames	Annotation rules stable; ambiguous cases documented
Public data preparation	RDD2022 four-class conversion	Conversion validator passes
Dataset V1	Custom 600–800 frames	Temporal split manifest produced
Baseline detector	YOLOv8s/n	Train/val/test pipeline reproducible
Proposed detector	YOLO26s/n	Same split/budget protocol as baseline
Dataset V2	1,200–2,000 custom frames recommended	Hard negatives + active-learning examples added
Tracking benchmark	Consecutive annotated clips	GT track IDs available in MOT/canonical format
Tracker experiment	ByteTrack vs BoT-SORT	IDSW/IDF1/HOTA and FPS reported
Counting	Event aggregation module	UCA + event-F1 measured

Milestone	Output	Acceptance criterion
Severity	Visual-extent module	Perspective-zone rule frozen before test
Integration	CLI pipeline	Video → MP4 + CSV + JSON completes
Full evaluation	Results tables/plots	Test set untouched until final evaluation
Academic package	Report + presentation + repo	Reproducibility checklist passes

Essential pre-project files

These files should exist **before substantial Codex-generated implementation**. Their purpose is to stop the architecture, labels and experiment definitions from silently changing as code grows.

File set

File	Purpose	Required sections / keys	Minimal template content
<code>rules.md</code>	The project's non-negotiable engineering and research rules. This prevents accidental label changes, leakage, hard-coded paths, undocumented experiments and false severity claims.	Data rules; architecture rules; experiment rules; coding rules; output rules; privacy rules	"Never random-split adjacent frames. Track IDs are not equivalent to physical damage IDs. Raw data is read-only."
<code>workflow.md</code>	Defines the exact order in which work happens so that application development does not begin before data and evaluation foundations exist.	Setup; inspect; sample; annotate; validate; train; evaluate; active learning; tracking; integration; final test	<code>inspect → sample → annotate → validate → train → evaluate → mine errors → retrain → track → aggregate → report</code>
<code>techstack.md</code>	Freezes dependencies and explains why each tool exists, reducing unnecessary framework switching.	Runtime; ML; video; annotation; evaluation; UI; testing; dependency policy	Python + Ultralytics/ PyTorch + OpenCV + FFmpeg + CVAT + TrackEval + pytest

File	Purpose	Required sections / keys	Minimal template content
<code>dataset_guidelines.md</code>	Becomes the human annotation handbook and dataset-quality specification.	Taxonomy; inclusions/exclusions; hard negatives; ambiguity; QA; splits; naming; privacy	"D40 requires visible pavement damage; dark region alone is not sufficient."
<code>annotation_schema.json</code>	Provides a machine-readable canonical label contract independent of YOLO's simplified text representation.	Schema version; classes; frame metadata; bbox; track ID; flags; QA status	Example provided below
<code>sampling_strategy.md</code>	Prevents arbitrary frame extraction and documents separate sampling policies for detector training and tracking evaluation.	Candidate sampling; dedup; active learning; tracking sampling; guard bands; quotas	"Initial candidate pool = ~1 FPS; tracking GT uses consecutive frames."
<code>repo_structure.md</code>	Defines where code, configuration, immutable data, experiments and outputs belong.	Directory tree; ownership; generated files; ignored files; naming	<code>src/</code> , <code>configs/</code> , <code>data/</code> , <code>experiments/</code> , <code>outputs/</code> , <code>tests/</code> , <code>docs/</code>
<code>testing_plan.md</code>	Makes the system testable beyond mAP. It covers data validators, aggregation logic, end-to-end smoke tests and full-video acceptance tests.	Unit; data; integration; model; regression; full-video test	"No bbox may lie outside image bounds; no source event may cross splits."
<code>ethics_and_license.md</code>	Records dataset licenses, teacher-video permissions, privacy handling and limitations of AI-generated assessments.	Data ownership; RDD license; Ultralytics license; privacy; publication; limitations	"Teacher footage private until written/publication permission is recorded."

Recommended tech-stack decision

Layer	Recommended V1	Alternative	Decision rationale
Language	Python	—	Single-language ML/video stack
Detector	YOLO26s	YOLO26n	Proposed detector
Baseline	YOLOv8s	YOLOv8n	Matched-scale comparison
Tracker	BoT-SORT	ByteTrack	Proposed vs baseline comparison
Deep learning	Ultralytics / PyTorch	Custom PyTorch	Lower implementation overhead
Annotation	CVAT	Label Studio	Video track support is valuable
Video decode/encode	FFmpeg + OpenCV	OpenCV only	Reliable encode/decode + frame processing
Tracking evaluation	TrackEval	Custom metrics	Standard ID/HOTA metrics
Data analysis	NumPy/Pandas/Matplotlib	—	Reporting
Configuration	YAML/JSON	TOML	Human-readable run configs
Tests	pytest	unittest	Simple automated tests
Dashboard	Streamlit, late-stage	FastAPI + custom frontend	Demo rather than core research
Output	CSV + JSON + MP4	Parquet optional	Human + machine readable

Ultralytics supports both YOLO26 and YOLOv8 through a common training/inference interface and provides built-in support for BoT-SORT and ByteTrack. CVAT supports manual video tracks, interpolation between keyframes, MOT output, COCO output and Ultralytics-YOLO export. ¹²

Example `rules.md`

Project Rules

Research integrity

- Never report a metric without naming the exact dataset/split used.
- Never use validation or test data for training or active learning.
- Never describe visual severity as physical depth, volume, or engineering severity.
- Track IDs are tracker outputs; they are NOT automatically physical damage IDs.

Dataset

- The canonical V1 classes are:
 - 0 / D00 / longitudinal_crack
 - 1 / D10 / transverse_crack
 - 2 / D20 / alligator_crack
 - 3 / D40 / pothole
- Do not add, remove, merge, or reorder classes without a schema-version change.
- Never randomly split adjacent frames from the same video.
-

All frames belonging to the same physical road-damage event must remain in one split.

- Raw video and raw annotations are immutable.

Experiments

- Every run must record:
 - git commit
 - dataset version
 - split manifest
 - model and weights
 - tracker config
 - image size
 - random seed
 - package versions
 - hardware
 - runtime
 - metrics
- Explicitly specify tracker configuration; never depend on framework defaults.

Code

- No absolute file paths.
- All tunable parameters belong in config files.
- Modules communicate through typed records/schemas.
- Unit-test counting, timestamps, bounding boxes and severity logic.
- Do not place research logic inside the Streamlit UI.

Outputs

- Every run receives a unique run_id.
- CSV and JSON counts must agree.
- Annotated video is a visualization; JSON/CSV is the analytical source of truth.

Privacy

- Do not publish teacher footage or extracted frames until permission is documented.
- Public demo material must blur clearly visible faces and registration plates where required.

One current-framework detail belongs in this file: tracker names must be explicitly configured. Ultralytics' current tracking documentation lists multiple trackers and identifies TrackTrack as the default when no tracker is passed, so relying on defaults would make the experiment vulnerable to library-version changes.

2

Example `annotation_schema.json`

YOLO labels alone should **not** be your canonical research record because the training format is deliberately minimal. CVAT's Ultralytics YOLO export supports detection boxes, but general annotation attributes are not represented in the same way; richer metadata should therefore remain in your canonical JSON/CVAT backup.

13

```
{
  "schema_version": "1.0.0",
  "taxonomy": "RDD2022_4CLASS",
  "classes": [
    {
      "id": 0,
      "code": "D00",
      "name": "longitudinal_crack"
    },
    {
      "id": 1,
      "code": "D10",
      "name": "transverse_crack"
    },
    {
      "id": 2,
      "code": "D20",
      "name": "alligator_crack"
    },
    {
      "id": 3,
      "code": "D40",
      "name": "pothole"
    }
  ],
  "example_frame": {
    "source_video": "teacher_dashcam_01.mp4",
    "frame_index": 18452,
    "timestamp_ms": 615067,
    "split": "train",
    "width": 1920,
    "height": 1080,
    "annotations": [
      {
        "annotation_id": "ann_000001",
```

```

        "class_id": 3,
        "bbox_xyxy": [810, 652, 1034, 824],
        "track_id": null,
        "physical_event_id": null,
        "occluded": false,
        "truncated": false,
        "ambiguous": false,
        "verified": true
    }
]
}
}

```

For the tracking benchmark, the same record becomes:

```

{
  "track_id": "GT_0042",
  "physical_event_id": "ROAD_DAMAGE_0042"
}

```

The distinction is intentional:

```

annotation_id
    = one box in one frame

track_id
    = temporal identity through a clip

physical_event_id
    = one actual road defect to count once

```

That separation is central to the research contribution.

Skeletons for the remaining files

`workflow.md`:

```

# Workflow

## Dataset workflow
Inspect video
→ sample candidate frames
→ deduplicate
→ annotate pilot

```

- review policy
- annotate V1
- validate
- freeze split v1

Training workflow

RDD2022 preparation

- YOLOv8 baseline
- YOLO26 experiment
- error analysis
- active-learning round
- retrain
- freeze detector

Video workflow

detector

- tracker
- smoother
- event aggregator
- severity
- report

Final evaluation

Freeze code/config/weights

- run untouched detection test
- run tracking benchmark
- run one-hour video
- generate tables/plots

techstack.md:

Technology Stack

Core

Python
Ultralytics
PyTorch
OpenCV
FFmpeg
NumPy
Pandas

Annotation

CVAT

Evaluation

```
Ultralytics validation
TrackEval
Matplotlib
```

Testing

```
pytest
```

Demo

```
Streamlit
```

Version policy

```
Pin exact versions in requirements-lock.txt.
Record CUDA, GPU and OS in run manifests.
```

dataset_guidelines.md:

Dataset Guidelines

Taxonomy

```
D00, D10, D20, D40
```

Positive examples

```
Define each class with visual examples.
```

Exclusions

```
Do not label:
```

- shadows
- puddles without visible damage
- manholes
- road markings
- colour changes alone
- normal repaired road unless included in a future taxonomy

Ambiguous examples

```
Mark ambiguous=true and exclude from final test until reviewed.
```

QA

```
Review 10–15% of annotations.
Run automated bbox/class/split validation.
```

sampling_strategy.md:

Sampling Strategy

Candidate pool

Start around 1 frame/second.

Diversity filtering

Remove near-duplicates and unusably blurred/dark frames.

Initial labels

600–800 frames.

Active learning

Mine false positives, false negatives and uncertain predictions.

Final target

1,200–2,000 high-quality custom detection frames.

Tracking benchmark

Use consecutive frames from selected clips.

Never use sparse detector-training sampling for tracking evaluation.

repo_structure.md:

```
road-damage-ai/
├─ README.md
├─ rules.md
├─ workflow.md
├─ techstack.md
├─ configs/
├─ data/
│   ├─ raw/
│   ├─ interim/
│   ├─ processed/
│   ├─ manifests/
│   └─ annotations/
├─ src/
├─ tests/
├─ scripts/
├─ experiments/
├─ models/
├─ outputs/
└─ docs/
    ├─ PROJECT_PRD.md
    ├─ dataset_guidelines.md
    ├─ sampling_strategy.md
    ├─ testing_plan.md
    └─ ethics_and_license.md
```

testing_plan.md:

Testing Plan

Data tests

Validate class IDs, bbox bounds, missing images and duplicate hashes.

Split tests

Verify no temporal block or physical_event_id exists in two splits.

Unit tests

Timestamp conversion

BBox conversion

Track confirmation

Smoothing

Event merging

Unique count

Severity zone

Integration

Run a 30–60 second fixture video.

Regression

The frozen fixture must retain expected event counts within defined tolerance.

Final acceptance

Process the full one-hour teacher video without crash.

ethics_and_license.md:

Ethics and Licensing

Teacher footage

Owner/redistribution permission: UNSPECIFIED.

Default status: private research material.

Public data

Record source, version, citation and license for every dataset.

Privacy

Do not publish identifiable faces or registration plates without review.

Model limitations

Outputs are AI predictions.

Visual severity is not engineering depth/volume assessment.

Deployment

Academic prototype only unless licensing and validation are separately resolved.

The official RoadDamageDetector project itself notes privacy handling by blurring clearly visible faces and vehicle license plates, which is a sensible precedent for public demonstrations of your own footage. The repository also states a CC BY-SA 4.0 license for its dataset images; preserve source attribution and verify the exact terms of every downloaded data component before redistribution. ¹⁴

Ultralytics currently offers YOLO under AGPL-3.0 and Enterprise licensing. Its own licensing guidance specifically includes university coursework/academic research as suitable for AGPL use when the project can be fully open-sourced, while proprietary/closed deployments require different licensing treatment. That should be recorded rather than left until submission time. ¹⁵

Dataset and annotation plan

Dataset layers

The project should deliberately maintain **three distinct datasets**.

Dataset ID	Contents	Purpose
D_PUBLIC	RDD2022 four-class labeled training data	Generic road-damage learning
D_CUSTOM_DET	Sparse, diverse labeled teacher-video frames	Local domain adaptation + detector evaluation
D_CUSTOM_TRACK	Consecutive video clips with GT track/event IDs	Tracking and counting evaluation

RDD2022's labeled data is available as Pascal VOC-style XML annotations, while its challenge test data is image-only. The project should therefore create its own deterministic train/development split from the labeled public portion while reserving the custom temporal test set as the main local-domain result. ¹⁶

Class policy

Use this taxonomy unless the video audit demonstrates that some categories are entirely absent:

Code	Definition for this project
D00	Predominantly longitudinal crack running approximately with the road direction
D10	Predominantly transverse crack running across the road direction
D20	Interconnected/alligator/fatigue crack network
D40	Visibly damaged pothole/depression region

These correspond directly to the four RDD2022 categories described by the dataset authors. 3

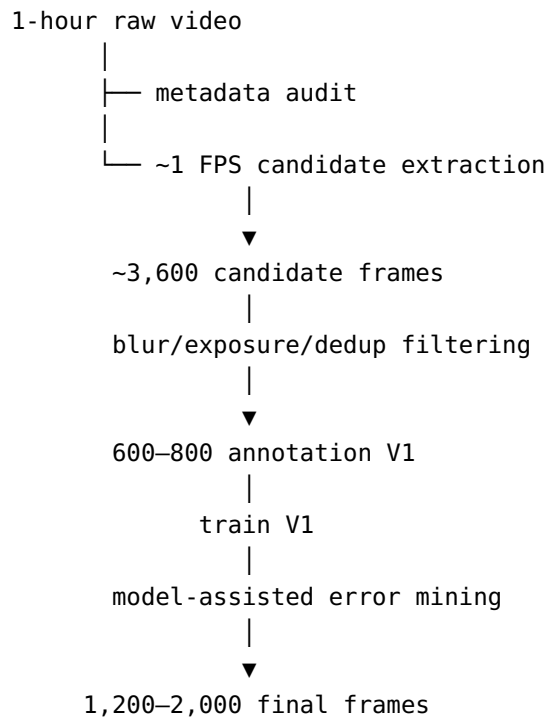
If a custom-video class is absent:

```
DO:
train it from RDD2022
retain the class mapping
report custom-video class support as N/A

DO NOT:
invent examples
relabel unrelated damage
claim local test performance for an absent class
```

Detection-frame sampling

Recommended starting process:



At one frame per second, a one-hour video produces about 3,600 initial candidates irrespective of the source video's original FPS. This is a **candidate-selection rate**, not the inference/tracking rate.

Recommended final custom detection dataset:

Category	Recommended range
Initial manually reviewed set	600–800 frames
Final detection set	1,200–2,000 frames
Negative/no-damage frames	~15–25% of custom images
Minimum desired custom instances per <i>present</i> class	~300, preferably more
Annotation QA sample	10–15%
Active-learning rounds	2–3

These are planning ranges rather than dataset-theory constants; the actual class distribution is currently **unspecified** and should determine the final allocation.

Hard-negative strategy

Hard negatives are particularly important because the road scene contains structures that resemble pavement damage. The LinkedIn RoadGuardian discussion itself included observations about wet areas/ reflections creating potential false detections; that feedback is anecdotal rather than scientific evidence, but it illustrates a realistic failure mode worth explicitly testing. ¹

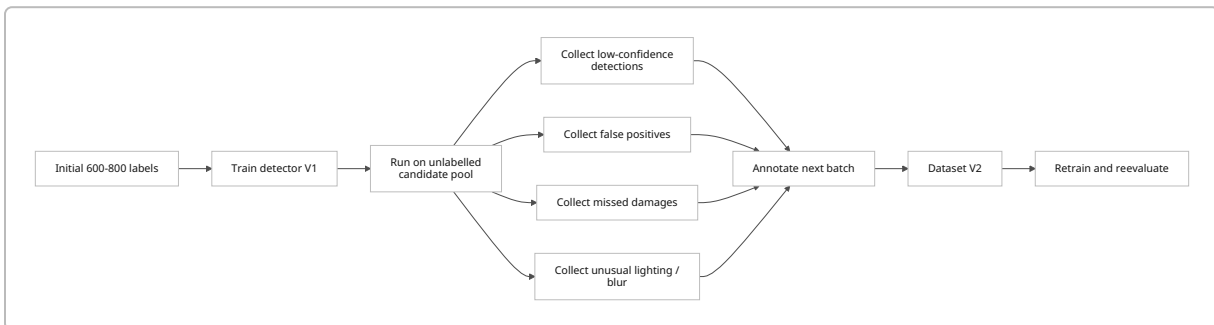
Maintain a hard-negative taxonomy such as:

Negative category	Action
Water / puddles	Include in no-damage frames
Wet asphalt	Include
Dark shadows	Include
Manholes/drain covers	Include
Tar joints	Include
Road paint	Include
Tyre marks	Include
Speed breakers	Include
Normal repair patches	Include as negatives for V1
Gravel/dirt	Include
Vehicle shadows	Include
Sun glare/reflection	Include

During active learning, every recurring false-positive category should receive an `error_category` in the error-analysis file rather than merely being visually inspected.

Active-learning loop

Use the detector to tell you **what to label next**:



The loop should operate only over training/development material. **The final temporal test blocks must never become active-learning candidates.**

Split rules

Random image splitting is prohibited.

Instead divide the route/video into contiguous temporal blocks:

```
video
|
├─ Block A ──── train
├─ Block B ──── train
├─ Block C ──── validation
├─ Block D ──── train
├─ Block E ──── test
├─ Block F ──── validation
└─ Block G ──── test
```

A 70/15/15 train/validation/test target is reasonable, but the exact assignment should be **group-aware rather than simply first 70%, next 15%, last 15%**. Entire route blocks should be assigned to a split while approximately maintaining class representation.

The following invariant should be automatically tested:

```
physical_event_id ∈ exactly one of {train, val, test}
source_temporal_block ∈ exactly one of {train, val, test}
```

A short temporal guard band—recommended approximately 2–5 seconds depending on vehicle speed and visibility duration—should be left between differently assigned blocks if a defect could otherwise appear on both sides.

Keeping correlated video frames inside the same split is consistent with leakage-prevention procedures used in video-machine-learning studies, where entire source videos or correlated frame groups are isolated to a single train/validation/test partition. ¹⁷

Tracking and counting ground truth

Do not attempt to manually track the whole one-hour video.

Create a **5–10 minute tracking benchmark** composed of several diverse clips:

```
Clip A: normal daylight
Clip B: dense damage
Clip C: shadows/water
Clip D: traffic/occlusion
Clip E: vibration/rough road
```

Actual available scenarios are currently unspecified and must come from the video audit.

Annotate those clips in CVAT **Track mode**. CVAT can interpolate tracks between manually chosen keyframes and supports MOT-style exports with frame IDs, track IDs and bounding boxes. ¹⁸

A practical annotation rate is:

```
tracking benchmark:
10–15 FPS working rate initially

not:
1 FPS sparse training rate
```

If the original footage is 30 FPS, it is acceptable to create a controlled 10/15 FPS tracking benchmark as long as **both trackers use precisely the same frame sequence**.

Each physical defect receives:

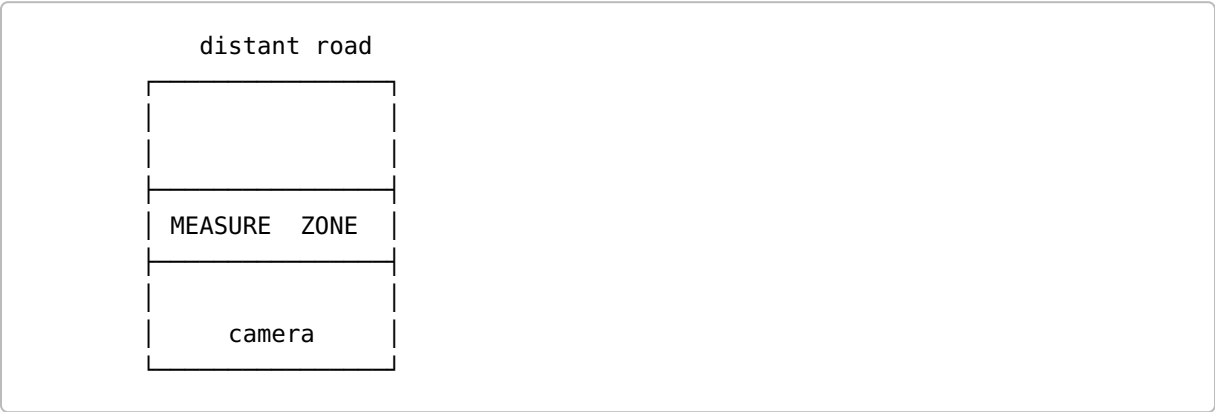
```
GT track_id
GT physical_event_id
GT class
first_visible_frame
last_visible_frame
optional occlusion flags
```

This is what makes ID switches and unique-count accuracy scientifically measurable.

Visual severity / extent policy

The system should record visual severity at a consistent **measurement zone** rather than using the maximum observed box size.

Conceptually:



For a confirmed track, use the observation closest to the center of this zone.

A better V1 design than raw common bbox-area thresholds is:

Class	Suggested visual extent feature
D00	Normalized crack-box major dimension
D10	Normalized crack-box major dimension
D20	Box area / road-ROI area
D40	Box area / road-ROI area

Thresholds for low / medium / high should be determined **only from training/development data**, frozen, and then applied to the test set.

The output label should say:

Visual severity or preferably **visual extent tier**

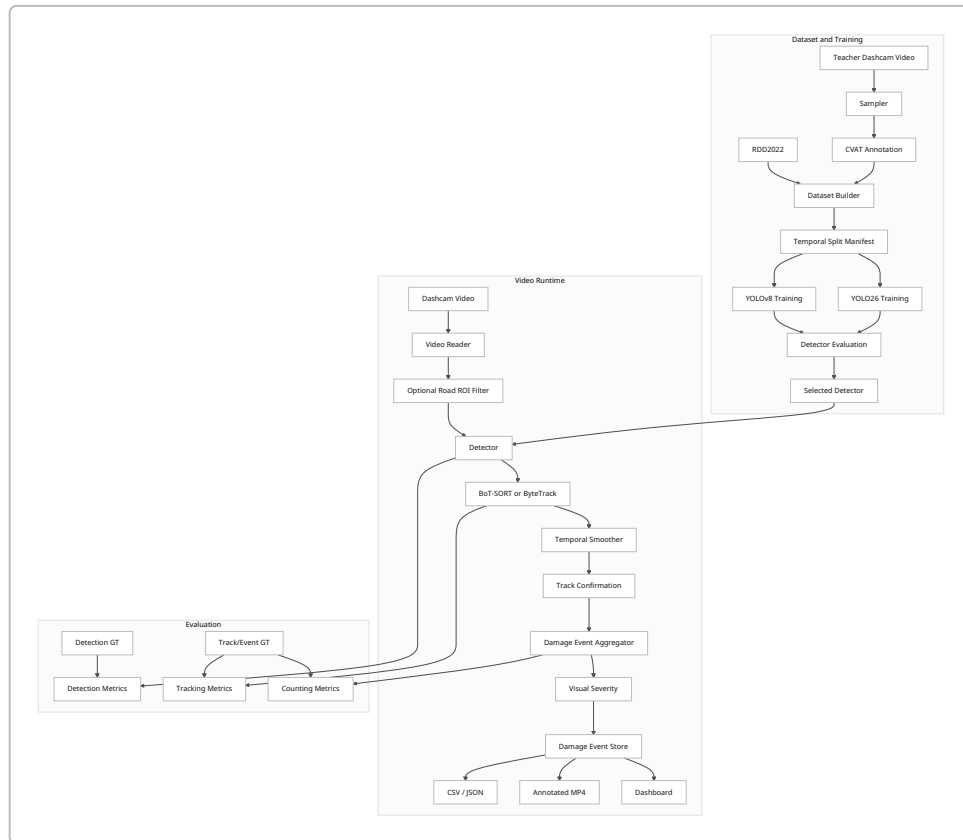
not:

pothole depth, structural severity, physical size or repair volume.

Your prior research correctly notes that precise depth/volume assessment from monocular footage without calibration or additional sensors is beyond the reliable scope of this project. filecite turn0file0

Architecture, repository modules and interfaces

The architecture should deliberately separate **training**, **runtime video processing**, and **evaluation**.



Canonical internal records

All modules should exchange defined data records instead of arbitrary dictionaries.

DetectionRecord

```
frame_index
timestamp_ms
class_id
confidence
bbox_xyxy
```

TrackObservation

```
DetectionRecord
+
track_id
```

```
track_age
tracker_state
```

DamageEvent

```
event_id
class_id
first_seen_ms
last_seen_ms
representative_timestamp_ms
representative_bbox
observation_count
mean_confidence
visual_severity
source_track_ids[]
```

The final field is important because **one physical event may eventually contain more than one tracker ID** after event-level merging.

Module contract

Module	Responsibility	Input	Output	Minimal interface
<code>video_inspector</code>	Read metadata/ quality	Video path	<code>VideoProfile</code>	<code>inspect_video(path)</code>
<code>frame_sampler</code>	Create candidate training frames	Video + sampling config	Frame manifest	<code>sample_video(...)</code>
<code>dataset_validator</code>	Validate labels/splits	Dataset manifest	QA report	<code>validate_dataset(...)</code>
<code>rdd_converter</code>	Convert Pascal VOC → canonical/ YOLO	RDD XML	Dataset	<code>convert_rdd2022(...)</code>
<code>detector</code>	Run YOLO prediction	Frame	<code>DetectionRecord[]</code>	<code>detect(frame)</code>
<code>tracker</code>	Associate detections	Frame + detections	<code>TrackObservation[]</code>	<code>update(...)</code>

Module	Responsibility	Input	Output	Minimal interface
<code>smoother</code>	Smooth confidence/ bbox/ persistence	Tracks	Smoothed tracks	<code>smooth(...)</code>
<code>track_confirmer</code>	Reject one-frame noise	Tracks	Confirmed tracks	<code>confirm(...)</code>
<code>event_aggregator</code>	Merge track observations into physical events	Confirmed tracks	<code>DamageEvent[]</code>	<code>update(...)</code>
<code>severity</code>	Compute visual tier	<code>DamageEvent</code> + ROI	Updated event	<code>estimate(event)</code>
<code>reporter</code>	Serialize results	Events	CSV/JSON	<code>write_reports(...)</code>
<code>renderer</code>	Annotate frames/video	Frame + tracks/events	MP4	<code>render(...)</code>
<code>detection_eval</code>	Compute detection metrics	GT + predictions	Metrics	<code>evaluate_detection(...)</code>
<code>tracking_eval</code>	Compute ID/ HOTA metrics	GT + tracks	Metrics	<code>evaluate_tracking(...)</code>
<code>count_eval</code>	Compute event/count metrics	GT events + predictions	Metrics	<code>evaluate_counting(...)</code>

CLI contract

A single project CLI is preferable to many unrelated scripts:

```
python -m road_damage inspect \
  --video data/raw/teacher_dashcam.mp4

python -m road_damage sample \
  --video data/raw/teacher_dashcam.mp4 \
  --config configs/sampling.yaml

python -m road_damage dataset validate \
  --manifest data/manifests/dataset_v1.json

python -m road_damage train \
```

```
--model yolo26s \  
--config configs/train_yolo26s.yaml  
  
python -m road_damage evaluate detection \  
--run experiments/yolo26s_final/  
  
python -m road_damage evaluate tracking \  
--tracker botsort \  
--benchmark data/tracking_test/  
  
python -m road_damage pipeline run \  
--video data/raw/teacher_dashcam.mp4 \  
--config configs/final_pipeline.yaml  
  
python -m road_damage report \  
--run outputs/run_2026_xxx/
```

Ultralytics supports Python and CLI interfaces for training and inference and provides tracker configuration files that can be selected at runtime, so your wrapper should expose those parameters rather than burying them in code. ¹⁹

Event aggregation and counting

The counting research should implement increasingly stronger algorithms.

Baseline A — frame count

Every detection = new damage

Expected result: severe over-counting.

Baseline B — raw track-ID count

Every tracker ID = one damage

Better, but an ID reset creates duplicates.

Baseline C — confirmed-track count

track persists $\geq N_{\min}$
AND mean confidence $\geq$ threshold
→ count once

Proposed D — event-level counting

```
detection
  ↓
tracker
  ↓
track confirmation
  ↓
class consensus
  ↓
counting/measurement-zone crossing
  ↓
recent-track duplicate merge
  ↓
physical DamageEvent
```

For example:

```
def should_confirm(track) -> bool:
    return (
        track.observation_count >= min_hits
        and track.mean_confidence >= min_mean_conf
    )
```

A temporary ID reset should not immediately create a new `DamageEvent`. The aggregator can compare the new track against recently closed events using:

```
same dominant class
+ small temporal gap
+ compatible image trajectory
+ spatial proximity / overlap near the counting zone
```

The project should start with these interpretable heuristics before adding homography or learned ReID.

Temporal smoothing

RoadGuardian's temporal smoothing and frame-hold mechanism is worth retaining as an engineering feature. ¹

Keep three concepts separate:

Detection confidence smoothing
Bounding-box smoothing
Display hold buffer

The display hold should **never create analytical detections**. In other words:

visual persistence != model evidence

Otherwise an anti-flicker visualization feature could accidentally inflate track duration or counts.

A simple exponential box smoother is adequate:

$$B_t^{smooth} = \alpha B_t + (1 - \alpha) B_{t-1}^{smooth}$$

where the parameter is defined in configuration and frozen for the smoothing ablation.

Minimal experimental plan

The final academic project does not need dozens of models. A small, controlled experiment matrix is stronger than a large collection of inconsistent runs.

Experimental datasets

Define:

D1 = RDD2022 four-class labeled training data
D2 = custom detection train/validation temporal blocks
D3 = untouched custom detection temporal test
D4 = custom consecutive tracking/counting benchmark
V1 = entire teacher dashcam video

RDD2022's official labeled training set can serve as public-domain pretraining/fine-tuning data, while D3 provides the key domain-specific test result because the official RDD2022 challenge test labels are unavailable. ⁸

Detector experiment

Compare:

YOLOv8s
vs
YOLO26s

Fallback:

YOLOv8n
vs
YOLO26n

Do **not** compare YOLOv8n against YOLO26s .

Hold constant:

train/val/test manifests
input resolution
model scale
epoch budget
batch policy
data augmentation policy
hardware
evaluation thresholds
seed where practical

Log any architecture-specific optimizer/default differences rather than hiding them.

Ultralytics officially supports both model generations, while YOLO26 adds its newer end-to-end architecture and updated training design. ⁴

Expected table:

Model	Params	mAP50	mAP50-95	D00 AP	D10 AP	D20 AP	D40 AP	FPS
YOLOv8s								
YOLO26s								

Required plots:

Precision-recall curve
Per-class AP comparison

mAP50-95 vs FPS scatter
False-positive category chart

Tracker experiment

Freeze the winning detector and compare:

ByteTrack
vs
BoT-SORT

Do not change detector confidence independently between tracker runs unless confidence threshold itself is a separate experiment.

ByteTrack explicitly associates lower-confidence detections to recover true objects that would otherwise be discarded; BoT-SORT adds camera-motion compensation and can incorporate appearance information. Ultralytics likewise recommends BoT-SORT as a moving-camera starting point and ByteTrack as the fastest/simple baseline. ²⁰

Expected table:

Tracker	HOTA	IDF1	MOTA	IDSW	Fragmentations	IDSW/100 GT	FPS
ByteTrack							
BoT-SORT							

TrackEval supports HOTA, IDF1 and CLEAR MOT-family metrics and is appropriate for this benchmark once custom annotations are converted to MOT-compatible structure. ²¹

Smoothing ablation

Use the chosen detector + tracker.

S0 = no smoothing
S1 = confidence smoothing
S2 = bbox EMA smoothing
S3 = bbox smoothing + display frame hold

Analytical tracking results should primarily compare **S0** against analytical smoothing; the frame-hold buffer should be evaluated as a visualization feature and must not modify ground-truth event logic.

Expected table:

Variant	IDSW	Mean bbox jitter	Event F1	UCA	FPS
None					
Confidence					
Box EMA					
EMA + hold					

A useful bbox-stability measure is normalized center displacement relative to adjacent frames after compensating for overall track movement, or simpler variance around a fitted trajectory.

Counting experiment

This may ultimately be the project's most interesting experiment.

Compare:

```
C0 = frame-level count
C1 = number of raw track IDs
C2 = confirmed-track count
C3 = confirmed + counting zone
C4 = confirmed + zone + recent-track event merge
```

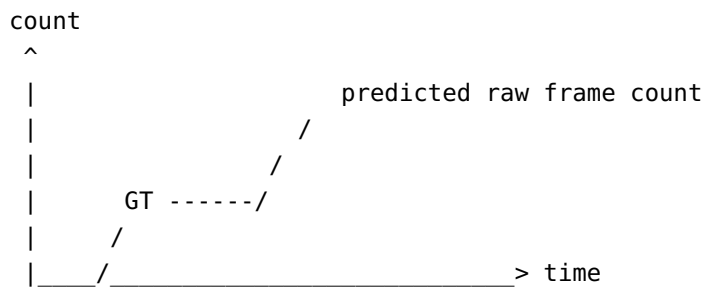
Expected table:

Counting algorithm	GT events	Predicted	UCA	Event precision	Event recall	Event F1	Duplicate rate
Frame count							
Raw IDs							
Confirmed IDs							
Zone							
Proposed event merge							

The most informative plot will be:

```
Cumulative unique damage count versus video time
```

Conceptually:



A successful aggregation algorithm should keep the predicted cumulative curve close to ground truth rather than climbing repeatedly for one persistent pothole.

Final experiment matrix

The project does **not** need the full Cartesian combination of every model/tracker/smoothing.

Use this efficient sequence:

```

Experiment A
YOLOv8 vs YOLO26
    ↓
select detector

Experiment B
ByteTrack vs BoT-SORT
    ↓
select tracker

Experiment C
smoothing ablation
    ↓
select smoothing

Experiment D
counting algorithms
    ↓
select event aggregator

Experiment E
final frozen pipeline
    ↓
untouched test + full 1-hour video

```

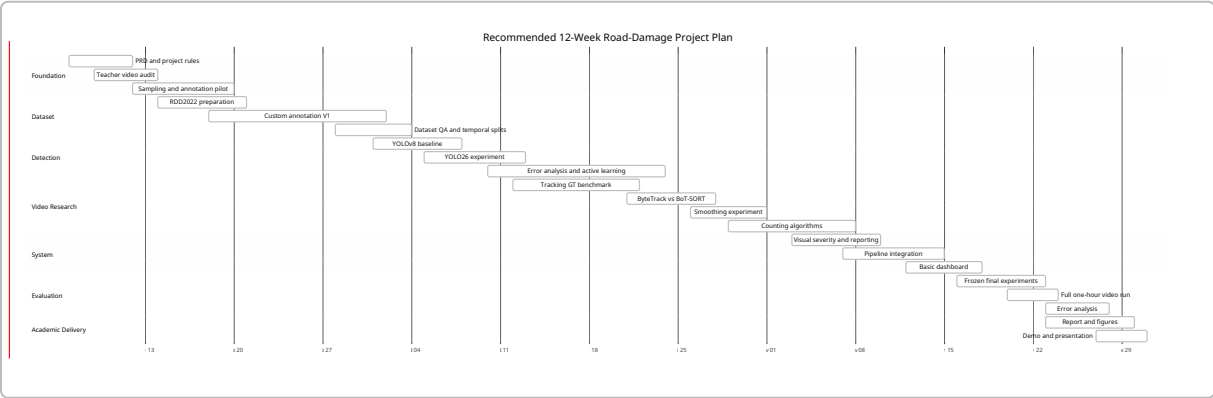
This ensures each experiment answers one research question.

The final results chapter should therefore contain approximately six core artifacts:

Artifact	Research question
Detector performance table	Is YOLO26 better than YOLOv8 here?
Per-class AP chart	Which damage types are difficult?
Tracker table	Which tracker preserves road-damage identity better?
Smoothing ablation table	Does temporal processing improve stability?
Counting table	Does event aggregation prevent over-counting?
mAP/FPS + cumulative-count plots	What is the accuracy-speed-system trade-off?

Timeline, resources and final execution plan

The academic deadline is **unspecified**, so the following is a recommended **twelve-week schedule starting in September 2026**, not an assumption about your submission date.



The highest-risk work is not the dashboard. It is:

annotation consistency

+

 temporal data leakage

+

 tracking ground truth

+

 unique-event aggregation

Those should therefore receive schedule buffer before UI work.

Resource budget

Actual hardware and available annotation time are currently **unspecified**.

Resource	Current status	Minimum workable	Recommended	Comfortable
GPU VRAM	Unspecified	~8 GB	12–16 GB	24 GB
System RAM	Unspecified	16 GB	16–32 GB	32+ GB
Working storage	Unspecified	~80–100 GB	150–250 GB	300+ GB
CPU	Unspecified	4–6 cores	6–8 cores	8+
Custom detection frames	Not yet created	~600	1,200–2,000	>2,000 only if useful
Tracking GT	Not yet created	~3 min	5–10 min	10–15 min
Annotation labor	Unspecified	~15–25 h	~25–60 h including QA	More only if dataset demands it
GPU training runs	Unspecified	4–6	~8–15 controlled runs	More for tuning

These resource values are planning estimates rather than model guarantees. Batch size must be chosen empirically after the actual GPU and image resolution are known.

RDD2022 itself is substantial—the official repository distributes multiple country-specific archives, including a particularly large Norway archive—so a project directory with extracted images, converted labels, training caches, checkpoints and output video benefits from significantly more space than the one-hour dashcam file alone. ¹⁴

Recommended final repository state before coding accelerates

```
road-damage-ai/
|
├─ README.md
├─ rules.md
├─ workflow.md
├─ techstack.md
|
├─ configs/
|   └─ sampling.yaml
|   └─ train_yolov8s.yaml
|   └─ train_yolo26s.yaml
|   └─ tracker_bytetrack.yaml
```

 create first
 create first
 create first

```

|   ├── tracker_botsort.yaml
|   └── final_pipeline.yaml
|
|   ├── data/
|   |   ├── raw/                # immutable
|   |   ├── public/
|   |   ├── interim/
|   |   ├── processed/
|   |   ├── annotations/
|   |   └── manifests/
|   |
|   ├── src/
|   |   ├── road_damage/
|   |   |   ├── video/
|   |   |   ├── datasets/
|   |   |   ├── detection/
|   |   |   ├── tracking/
|   |   |   ├── aggregation/
|   |   |   ├── severity/
|   |   |   ├── reporting/
|   |   |   ├── evaluation/
|   |   |   └── cli.py
|   |
|   ├── tests/
|   |   ├── unit/
|   |   ├── integration/
|   |   └── data/
|   |
|   ├── experiments/
|   |
|   ├── outputs/
|   |
|   └── docs/
|       ├── PROJECT_PRD.md      ✓ source of truth
|       ├── dataset_guidelines.md ✓
|       ├── annotation_schema.json ✓
|       ├── sampling_strategy.md ✓
|       ├── repo_structure.md   ✓
|       ├── testing_plan.md     ✓
|       └── ethics_and_license.md ✓

```

Definition of “project complete”

The project should not be considered finished merely because boxes appear on a video.

It is complete when a clean environment can execute a frozen configuration that:

```
teacher_dashcam.mp4
  ↓
validated video reader
  ↓
selected trained detector
  ↓
selected tracker
  ↓
temporal stabilization
  ↓
confirmed tracks
  ↓
unique DamageEvents
  ↓
visual severity / extent
  ↓
CSV + JSON + annotated MP4
  ↓
evaluation tables and plots
```

and the project can answer, with quantitative evidence:

How accurately can the system detect road damage?

Does YOLO26 improve the accuracy/speed trade-off over YOLOv8 on this dataset?

Does BoT-SORT reduce tracking identity failures relative to ByteTrack for moving-camera road footage?

Does temporal smoothing improve stability?

How much does event-level aggregation reduce repeated counting?

How accurately does the final system count unique physical road-damage events on held-out video?

That is the distinction between a **portfolio-style YOLO demonstration** and a defensible undergraduate AI/ML research project. The RoadGuardian LinkedIn system supplies useful inspiration for detector/tracker/smoothing engineering, while RDD2022 provides the canonical four-class public research basis, and your teacher's one-hour dashcam video supplies the domain-specific data needed to make the project genuinely yours. ²²

1 22 AI-Powered Road Hazard Detection with YOLO26 and Computer Vision | Areeba Shahid posted on the topic | LinkedIn

<https://lnkd.in/p/dhvUzTbu>

2 YOLO Multi-Object Tracking in Video | Ultralytics

https://docs.ultralytics.com/modes/track?utm_source=chatgpt.com

3 RDD2022: A multi-national image dataset for automatic Road Damage Detection

https://arxiv.org/abs/2209.08538?utm_source=chatgpt.com

4 12 19 Ultralytics YOLO26 | Ultralytics

https://docs.ultralytics.com/models/yolo26?utm_source=chatgpt.com

5 BoT-SORT: Robust Associations Multi-Pedestrian Tracking

https://arxiv.org/abs/2206.14651?utm_source=chatgpt.com

6 8 RDD2022: A multi-national image dataset for automatic road damage detection - Arya - 2024 - Geoscience Data Journal - Wiley Online Library

https://rmets.onlinelibrary.wiley.com/doi/10.1002/gdj3.260?utm_source=chatgpt.com

7 Temporally Coherent Embeddings for Self-Supervised Video Representation Learning

https://arxiv.org/abs/2004.02753?utm_source=chatgpt.com

9 20 ByteTrack: Multi-Object Tracking by Associating Every Detection Box

https://arxiv.org/abs/2110.06864?utm_source=chatgpt.com

10 21 GitHub - JonathonLuiten/TrackEval: HOTA (and other) evaluation metrics for Multi-Object Tracking (MOT). · GitHub

https://github.com/JonathonLuiten/TrackEval?utm_source=chatgpt.com

11 MOT | CVAT

https://docs.cvat.ai/docs/manual/advanced/formats/format-mot/?utm_source=chatgpt.com

13 Ultralytics YOLO | CVAT

https://docs.cvat.ai/docs/dataset_management/formats/format-yolo-ultralytics/?utm_source=chatgpt.com

14 16 GitHub - sekilab/RoadDamageDetector · GitHub

https://github.com/sekilab/RoadDamageDetector?utm_source=chatgpt.com

15 Ultralytics License

https://www.ultralytics.com/license?utm_source=chatgpt.com

17 Development of Machine Learning Copilot to Assist Novices in Learning Flexible Laryngoscopy - PMC

https://pmc.ncbi.nlm.nih.gov/articles/PMC11832335/?utm_source=chatgpt.com

18 Track mode | CVAT

https://docs.cvat.ai/docs/annotation/manual-annotation/modes/track-mode-basics/?utm_source=chatgpt.com